

Kubernetes Assignment 1 – Pod Creation, Management and Deletion

Step 1: minikube start

minikube start starts a local Kubernetes cluster on your computer, allowing you to create, manage, and test Kubernetes resources such as Pods, Deployments, and Services.

```
hp@Dharanidhar MINGW64 ~/gitrepository/te-5th_and_6th_batch_assignments/Pavithra/K8S-Assignment-1 (main)
$ minikube start
* minikube v1.38.1 on Microsoft Windows 11 Pro 25H2
* Using the docker driver based on existing profile
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.50 ...
* Updating the running docker "minikube" container ...
* Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

Step 2: minikube status

minikube status checks the current status of the Minikube Kubernetes cluster and confirms whether it is running and ready to use.

```
hp@Dharanidhar MINGW64 ~/gitrepository/te-5th_and_6th_batch_assignments/Pavithra/K8S-Assignment-1 (main)
$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

Step 3: kubectl get nodes

kubectl get nodes displays the nodes available in the Kubernetes cluster and shows whether they are ready to run Pods.

```
hp@Dharanidhar MINGW64 ~/gitrepository/te-5th_and_6th_batch_assignments/Pavithra/K8S-Assignment-1 (main)
$ kubectl get nodes
NAME        STATUS    ROLES          AGE      VERSION
minikube    Ready     control-plane   2d15h    v1.35.1
```

Step 4: Create a Pod YAML File

The vim pod1.yaml command is used to create a new YAML file or open an existing YAML file for editing using the Vim text editor.

The ls command is used to list all files and folders in the current directory.

The cat pod1.yaml command is used to **display the contents of the pod1.yaml file** on the terminal.

```
hp@Dharanidhar MINGW64 ~/gitrepository/te-5th_and_6th_batch_assignments/Pavithra/K8S-Assignment-1 (main)
$ vim pod1.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: app1
  labels:
    app1: web-application
spec:
  containers:
    - name: nginx-container
      image: nginx
      ports:
        - containerPort: 80
```

```
hp@Dharanidhar MINGW64 ~/gitrepository/te-5th_and_6th_batch_assignments/Pavithra/K8S-Assignment-1 (main)
$ ls
pod1.yaml

hp@Dharanidhar MINGW64 ~/gitrepository/te-5th_and_6th_batch_assignments/Pavithra/K8S-Assignment-1 (main)
$ cat pod1.yaml
apiVersion: v1
kind: Pod
metadata:
  name: app1
  labels:
    app1: web-application
spec:
  containers:
  - name: nginx-container
    image: nginx
    ports:
    - containerPort: 80
```

Step 5: Apply the YAML File

kubectl apply -f pod1.yaml reads the pod1.yaml file and creates or updates the Pod in the Kubernetes cluster according to the configuration defined in the file.

```
hp@Dharanidhar MINGW64 ~/gitrepository/te-5th_and_6th_batch_assignments/Pavithra/K8S-Assignment-1 (main)
$ kubectl apply -f pod1.yaml
pod/app1 created
```

Step 6: Verify the Pod

kubectl get pods displays all the Pods in the current Kubernetes namespace and shows their status, readiness, restart count, and age.

```
hp@Dharanidhar MINGW64 ~/gitrepository/te-5th_and_6th_batch_assignments/Pavithra/K8S-Assignment-1 (main)
$ kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
app1      1/1     Running   1 (2m55s ago)    5m
```

Step 7: view detailed information

kubectl describe pod app1 displays detailed information about the app1 Pod, including its configuration, current status, container details, and events. It is commonly used to troubleshoot Pod-related issues.

```
hp@Dharanidhar MINGW64 ~/gitrepository/te-5th_and_6th_batch_assignments/Pavithra/K8S-Assignment-1 (main)
$ kubectl describe pod app1
Name:      app1
Namespace: default
Priority:   0
Service Account: default
Node:      minikube/192.168.49.2
Start Time: Mon, 03 Aug 2026 12:54:05 +0530
Labels:    app1=web-application
Annotations: <none>
Status:    Running
IP:        10.244.0.7
IPs:       10.244.0.7
Containers:
  nginx-container:
    Container ID:   docker://924d760dc2b389d3b4d449b6b8da26a2c7dff5a54a5321d77329e9cc166157b5
    Image:          nginx
    Image ID:       docker-pullable://nginx@sha256:5a88c9c45479443d7be2eadc894b4ed0a9801bae03d97a5760ae13b5c2
    Port:          80/TCP
    Host Port:     0/TCP
    State:         Running
      Started:     Mon, 03 Aug 2026 12:56:28 +0530
    Last State:    Terminated
      Reason:      Completed
      Exit Code:    0
      Started:     Mon, 03 Aug 2026 12:54:19 +0530
      Finished:    Mon, 03 Aug 2026 12:56:10 +0530
    Ready:         True
    Restart Count: 1
    Environment:   <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-wnmf (ro)
Conditions:
  Type              Status
  PodReadyToStartContainers  True
  Initialized        True
  Ready              True
  ContainersReady    True
  PodScheduled       True
Volumes:
  kube-api-access-wnmf:
    Type:      Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName: kube-root-ca.crt
    Optional:    false
    DownwardAPI: true
```

Step 8: view pod logs

kubectl logs app1 displays the log messages generated by the application running inside the app1 Pod. It is commonly used to monitor the application and troubleshoot errors.

```
hp@Dharanidhar MINGW64 ~/gitrepository/te-5th_and_6th_batch_assignments/Pavithra/K8S-Assignment-1 (main)
$ kubectl logs app1
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/08/03 07:26:28 [notice] 1#1: using the "epoll" event method
2026/08/03 07:26:28 [notice] 1#1: nginx/1.31.3
2026/08/03 07:26:28 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/08/03 07:26:28 [notice] 1#1: OS: Linux 6.18.33.2-microsoft-standard-WSL2
2026/08/03 07:26:28 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/08/03 07:26:28 [notice] 1#1: start worker processes
2026/08/03 07:26:28 [notice] 1#1: start worker process 29
2026/08/03 07:26:28 [notice] 1#1: start worker process 30
2026/08/03 07:26:28 [notice] 1#1: start worker process 31
2026/08/03 07:26:28 [notice] 1#1: start worker process 32
2026/08/03 07:26:28 [notice] 1#1: start worker process 33
2026/08/03 07:26:28 [notice] 1#1: start worker process 34
2026/08/03 07:26:28 [notice] 1#1: start worker process 35
2026/08/03 07:26:28 [notice] 1#1: start worker process 36
```

Step 9: Accessing the pod shell

kubectl exec -it app1 -- sh opens an interactive shell inside the app1 Pod, allowing you to run Linux commands, inspect files, and troubleshoot the running container.

```
hp@Dharanidhar MINGW64 ~/gitrepository/te-5th_and_6th_batch_assignments/Pavithra/K8S-Assignment-1 (main)
$ kubectl exec -it app1 -- sh
# ls
bin boot dev docker-entrypoint.d docker-entrypoint.sh etc home lib lib64 media mnt opt proc root run sbin
# exit
```

Step 10: Delete the Pod

The kubectl delete pod app1 command is used to delete the Pod named app1 from the Kubernetes cluster.

```
hp@Dharanidhar MINGW64 ~/gitrepository/te-5th_and_6th_batch_assignments/Pavithra/K8S-Assignment-1 (main)
$ kubectl delete pod app1
pod "app1" deleted from default namespace
```

Pod Lifecycle: A Pod goes through different stages from creation to termination.

1. Pending : The Pod has been created, but it is still preparing to start.
2. Running: The Pod is running successfully, and the application is active.
3. Succeeded : The Pod has completed its work successfully and stopped.
4. Failed : The Pod has stopped because of an error or failure.

CrashLoopBackOff:

CrashLoopBackOff means the container starts, crashes, Kubernetes restarts it, and it crashes again repeatedly. Kubernetes waits a little longer before each restart, which is called a back-off.

Reasons:

Application Errors, Memory issues, Resource Limits, Configuration Errors, Permission issues.

Troubleshoot:

1. Use `kubectl describe pod app1` to find error details and events.
2. Check application errors using `kubectl logs app1`
3. Verify the YAML file for incorrect configurations, images.
4. Fix the issue and redeploy the Pod using `kubectl apply -f pod1.yaml`.
5. Verify that the Pod is running successfully using `kubectl get pods`.